Recommender System – Opis implementacije

1. Opis implementacije recommender sistema

1.1 Svrha i ciljevi

Aplikacija Yamore uključuje integrisani sistem preporuke koji korisnicima predlaže jahte u skladu s njihovim preferencijama kako bi se poboljšalo korisničko iskustvo i angažman na platformi. Sistem koristi kombinaciju dva pristupa:

- Content-based filtering – preporuke zasnovane na atributima objekata (tip jahte, lokacija, usluge) i prethodnom ponašanju korisnika.
- Collaborative filtering signali – korištenje ocjena i popularnosti (npr. koliko puta je jahta rezervisana) za rangiranje preporuka.

Cilj sistema je prikazati svakom korisniku listu „Recommended for you“, koja odražava njegove prethodne rezervacije, ocjene i izabrane usluge (npr. skiper, catering), dok se istovremeno promovišu dobro ocijenjene i popularne jahte.

1.2 Podaci koji se koriste za preporuke

Sistem koristi sljedeće podatke:

Izvor podataka	Upotreba u algoritmu
Reservations	ID-jevi jahti koje je korisnik rezervisao (neotkazane rezervacije). Koriste se za određivanje preferiranih kategorija, lokacija i država.
Reviews	Ocjene koje je korisnik dao (npr. ≥ 4) kako bi se naglasile jahte slične onima koje je korisnik visoko ocijenio.
Reservation–Services	Usluge povezane s rezervacijama korisnika (npr. skiper, catering). Koristi se za preferiranje jahti koje nude iste ili slične usluge.
Yacht attributes	Kategorija, lokacija (grad), država i povezane usluge (YachtServices).
Popularity	Broj neotkazanih rezervacija po jahti (signal kolaborativne filtracije).
Average rating	Prosječna ocjena recenzija za određenu jahtu (signal kvaliteta).

Samo aktivne jahte (state = "active") se preporučuju. Jahte koje je korisnik već rezervisao se isključuju iz njegove personalizovane liste preporuka.

1.3 Logika algoritma (Backend)

Logika je implementirana u backend servisu i može se opisati kroz sljedeće korake.

Korak 1 – Profil korisnika (za prijavljenog korisnika sa historijom)

Iz baze podataka izračunavaju se:

- Preferirane kategorije – različiti ID-jevi kategorija jahti koje je korisnik rezervisao (neotkazane).
- Preferirane lokacije – različiti ID-jevi gradova tih jahti.
- Preferirane države – različiti ID-jevi država tih lokacija.
- Preferirane usluge – različiti ID-jevi usluga povezanih sa rezervacijama korisnika (npr. skiper, catering).

Korak 2 – Skup kandidata

Kandidati su sve aktivne jahte koje korisnik nije već rezervisao (isključujući userReservationYachtIds).

Korak 3 – Bodovanje i sortiranje (personalizacija)

Kada korisnik ima barem jednu preferiranu kategoriju, lokaciju ili državu:

Content-based kriteriji

1. Prednost imaju jahte čija kategorija odgovara korisnikovim preferencijama.
2. Zatim se preferiraju jahte u istoj lokaciji (gradu).
3. Nakon toga jahte u istoj državi.
4. Zatim jahte koje nude barem jednu od korisnikovih preferiranih usluga (putem YachtServices).

Collaborative / kvalitet

5. Zatim se rezultati sortiraju po prosječnoj ocjeni (veća ocjena ima prednost).
6. Nakon toga po broju neotkazanih rezervacija (popularnije jahte imaju prednost).

Ovo je implementirano kao višeslojno sortiranje:

kategorija → lokacija → država → usluge → prosječna ocjena → broj rezervacija.

Korak 4 – Korisnici bez historije

Ako korisnik nema rezervacija (ili nema izvedene kategorije/lokacije/države), lista nije personalizovana. U tom slučaju jahte se sortiraju samo prema:

- popularnosti (broj rezervacija)
- prosječnoj ocjeni

Korak 5 – Neprijavljeni korisnici

Ako korisnik nije identificiran (npr. nema userId), koristi se isti fallback pristup:

sve aktivne jahte se sortiraju prema broju rezervacija i zatim prema prosječnoj ocjeni.

Korak 6 – Odgovor sistema

Backend vraća straničenu listu (paged list) objekata tipa YachtOverviewDto koji sadrže osnovne informacije:

- id
- naziv jahte
- lokaciju
- državu
- cijenu
- ocjenu

Na taj način klijentska aplikacija može prikazati isti card layout kao u glavnoj listi jahti.

1.4 Integracija API-ja i klijentske aplikacije

- Endpoint:
  GET /Yachts/recommendations
- Parametri:
  userId, page, pageSize
- Autentifikacija:
  Ako klijent ne pošalje userId, server može uzeti ID prijavljenog korisnika iz tokena.
- Klijent:
  Flutter aplikacija poziva ovaj endpoint prilikom učitavanja home stranice (sa ID-jem prijavljenog korisnika). Server vraća listu preporučenih jahti koje se prikazuju u sekciji „Recommended for you“.

Ako API vrati praznu listu, aplikacija koristi fallback listu (npr. najskuplje ili najpopularnije jahte), kako sekcija ne bi bila prazna.

1.5 Sažetak implementacije

Aspekt	Implementacija
Content-based filtracija	kategorija, lokacija, država i usluge iz korisnikovih rezervacija
Collaborative filtracija	prosječna ocjena i broj rezervacija

Aspekt	Implementacija
Učenje sistema	što korisnik više rezerviše i ocjenjuje, preporuke postaju preciznije
Prikaz	sekcija “Recommended for you” na početnoj stranici

2. Putanja i source code glavne logike

Backend – glavna logika preporuke

Putanja datoteke:

Yamore/Yamore.Services/Services/YachtsService.cs

Glavna logika nalazi se u metodi: **GetRecommendations** (linije **173–251**) i pomoćnoj metodi:

BuildRecommendationOverviewResult (linije **253–283**)

Ova datoteka sadrži:

- izgradnju korisničkog profila
- pronalazak kandidata
- sortiranje prema content-based kriterijima
- sortiranje prema ocjenama i popularnosti
- mapiranje rezultata u YachtOverviewDto

YachtsService.cs - metoda **GetRecommendations**

```
2 references | Zerina Ibrulj, 19 hours ago | 1 author, 2 changes
public PagedResponse<YachtOverviewDto> GetRecommendations(int? userId, int page = 0, int pageSize = 10)
{
    var activeYachts = Context.Yachts
        .Include(y => y.Owner)
        .Include(y => y.Location).ThenInclude(c => c.Country)
        .Include(y => y.Reviews)
        .Include(y => y.YachtServices)
        .Include(y => y.Reservations)
        .Where(y => y.StateMachine == "active")
        .AsQueryable();

    IOrderedQueryable<Database.Yacht> ordered;

    if (userId.HasValue)
    {
        // User profile from past reservations (confirmed) and positive reviews (rating >= 4)
        var userReservationYachtIds = Context.Reservations
            .Where(r => r.UserId == userId && r.Status != "Cancelled")
            .Select(r => r.YachtId).Distinct().ToList();
```

```
if (userId.HasValue)
{
    // User profile from past reservations (confirmed) and positive reviews (rating >= 4)
    var userReservationYachtIds = Context.Reservations
        .Where(r => r.UserId == userId && r.Status != "Cancelled")
        .Select(r => r.YachtId).Distinct().ToList();

    var highlyRatedYachtIds = Context.Reviews
        .Where(r => r.UserId == userId && r.Rating >= 4)
        .Select(r => r.YachtId).Distinct().ToList();

    var preferredCategoryIds = Context.Yachts
        .Where(y => userReservationYachtIds.Contains(y.YachtId))
        .Select(y => y.CategoryId).Distinct().ToList();

    var preferredLocationIds = Context.Yachts
        .Where(y => userReservationYachtIds.Contains(y.YachtId))
        .Select(y => y.LocationId).Distinct().ToList();

    var preferredCountryIds = Context.Yachts
        .Where(y => userReservationYachtIds.Contains(y.YachtId) && y.Location != null)
        .Select(y => y.Location!.CountryId).Distinct().ToList();
```

```
var preferredCountryIds = Context.Yachts
    .Where(y => userReservationYachtIds.Contains(y.YachtId) && y.Location != null)
    .Select(y => y.Location!.CountryId).Distinct().ToList();

// Services the user often added to reservations (skipper, catering, etc.)
var preferredServiceIds = Context.ReservationServices
    .Where(rs => rs.Reservation != null && rs.Reservation.UserId == userId)
    .Select(rs => rs.ServiceId).Distinct().ToList();

// Content-based + collaborative: exclude already-booked, prefer same category/location/country and yachts offering
var candidates = activeYachts
    .Where(y => !userReservationYachtIds.Contains(y.YachtId));

if (preferredCategoryIds.Count == 0 && preferredLocationIds.Count == 0 && preferredCountryIds.Count == 0)
{
    // No history: fall back to popular (collaborative - most booked, then by rating)
    ordered = candidates
        .OrderByDescending(y => y.Reservations.Count(r => r.Status != "Cancelled"))
        .ThenByDescending(y => y.Reviews.Any(r => r.Rating.HasValue) ? y.Reviews.Average(r => r.Rating ?? 0) : 0);
}
else
{
    // Score: prefer category match, then location, then country; then order by rating and popularity
    ordered = candidates
        .OrderByDescending(y => preferredCategoryIds.Contains(y.CategoryId))
        .ThenByDescending(y => preferredLocationIds.Contains(y.LocationId))
        .ThenByDescending(y => y.Location != null && preferredCountryIds.Contains(y.Location.CountryId))
        .ThenByDescending(y => y.YachtServices.Any(ys => preferredServiceIds.Contains(ys.ServiceId)))
        .ThenByDescending(y => y.Reviews.Any(r => r.Rating.HasValue) ? y.Reviews.Average(r => r.Rating ?? 0) : 0)
        .ThenByDescending(y => y.Reservations.Count(r => r.Status != "Cancelled"));
}

var totalCount = ordered.Count();
var list = ordered.Skip(page * pageSize).Take(pageSize).ToList();
return BuildRecommendationOverviewResult(list, totalCount);
```

```
}
else
{
    // Anonymous: most popular active yachts by booking count, then rating
    ordered = activeYachts
        .OrderByDescending(y => y.Reservations.Count(r => r.Status != "Cancelled"))
        .ThenByDescending(y => y.Reviews.Any(r => r.Rating.HasValue) ? y.Reviews.Average(r => r.Rating ?? 0) : 0);
    var totalCount = ordered.Count();
    var list = ordered.Skip(page * pageSize).Take(pageSize).ToList();
    return BuildRecommendationOverviewResult(list, totalCount);
}
```

API endpoint

Putanja:

Yamore/Yamore.API/Controllers/YachtsController.cs

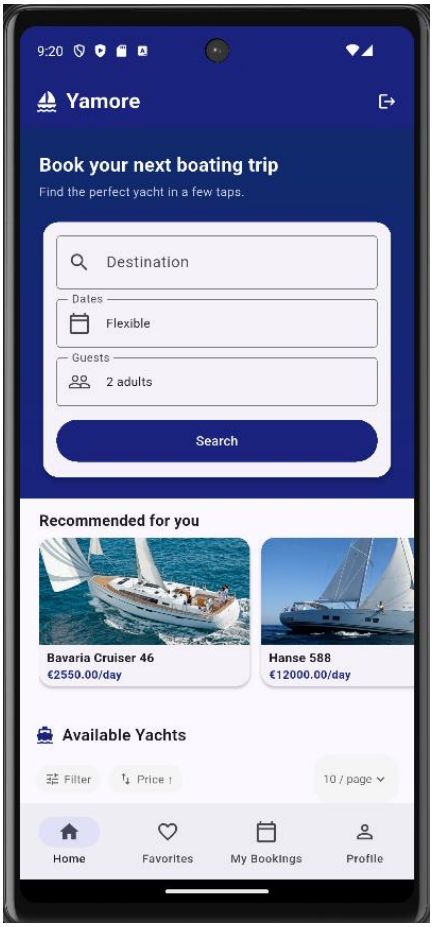
```
[HttpGet("recommendations")]
0 references | Zerina Ibrulj, 20 hours ago | 1 author, 2 changes
public PagedResponse<YachtOverviewDto> GetRecommendations([FromQuery] int? userId, [FromQuery] int page = 0, [Fr
{
    var currentUserId = User?.FindFirstValue(ClaimTypes.NameIdentifier);
    var id = userId ?? (int.TryParse(currentUserId, out var uid) ? (int?)uid : null);
    return _yachtsService.GetRecommendations(id, page, pageSize);
}
```

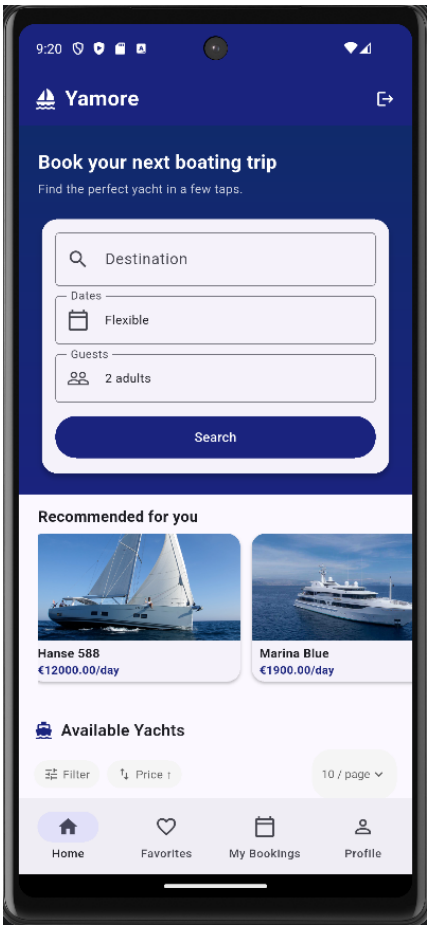
3. Putanja i screenshot iz pokrenute aplikacije

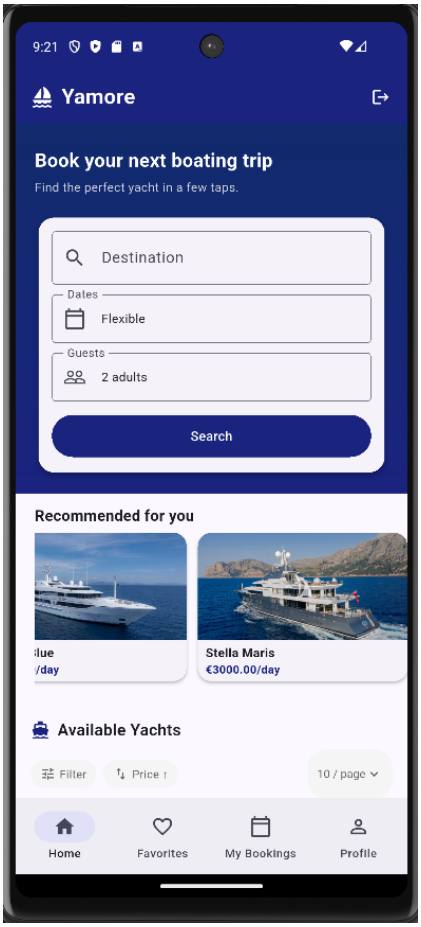
Preporuke se prikazuju na **početnoj stranici mobilne aplikacije** nakon prijave korisnika.

Sekcija se sastoji od:

- naslova **"Recommended for you"**
- horizontalne liste kartica jahti.







UI komponenta koja prikazuje preporuke nalazi se u datoteci:

Yamore/UI/yamore_mobile/lib/screens/mobile/mobile_home_tab.dart

Metoda:

_buildRecommendedStrip()

4. Sažetak putanja datoteka

Svrha	Putanja
Backend logika preporuke	Yamore/Yamore.Services/Services/YachtsService.cs
API endpoint	Yamore/Yamore.API/Controllers/YachtsController.cs
Flutter API poziv	yamore_mobile/lib/services/api_service.dart
Flutter prikaz preporuka	yamore_mobile/lib/screens/mobile/mobile_home_tab.dart